

GenAi Analysis

Name of AI #1:

Copilot

Prompt:

Write a C program that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check whether the guess is too high, too low, or correct. Stop the loop immediately if the user guesses correctly. Print a final win or lose message. Include comments explaining the code.

Generated Code:

```
#include <stdio.h>

int main() {
// Fixed secret number between 1 and 10
int secretNumber = 7;

// Variable to store the user's guess
int guess;

// Flag to track whether the user guessed correctly
int guessedCorrectly = 0;

// Allow the user up to 3 guesses
for (int attempt = 1; attempt <= 3; attempt++) {
// Prompt the user for a guess
printf("Attempt %d of 3: Enter your guess (1-10): ", attempt);
scanf("%d", &guess);

// Check the guess
if (guess == secretNumber) {
printf("Correct! You guessed the secret number.\n");
guessedCorrectly = 1;

// Stop the loop immediately if correct
break;
} else if (guess < secretNumber) {
printf("Too low! Try again.\n");
} else {
printf("Too high! Try again.\n");
}
}
```

```

// Print final win or lose message
if (guessedCorrectly) {
    printf("You win!\n");
} else {
    printf("You lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

Analysis:

The code is correct because it allows the user up to three guesses and correctly tells them if their guess is too high, too low, or correct. Its execution time is $O(1)$ because the loop can only run three times, and its space complexity is $O(1)$ because it only uses a few variables and does not need extra memory. The code is also easy to maintain because it has clear variable names and comments. However, it could be improved by adding input validation and using constants for values like the number of attempts. I will use copilot as my basis as it is the easiest to improve upon.

Improvements:

I added comments on each line, and i also added input validation like i discussed in my analysis

Name of AI #2:

Gemini

Prompt:

Write a C program that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check whether the guess is too high, too low, or correct. Stop the loop immediately if the user guesses correctly. Print a final win or lose message. Include comments explaining the code.

Generated Code:

```

#include <stdio.h> // Standard input/output library for printf and scanf

int main() {
    // Define the secret number and the maximum number of attempts
    const int SECRET = 7;
    const int MAX_ATTEMPTS = 3;

    int guess;
    int attempt = 1;

```

```

int guessed_correctly = 0; // Flag to track if the user won (0 = false, 1 = true)

printf("Welcome to the Number Guessing Game!\n");
printf("I'm thinking of a number between 1 and 10. You have %d tries.\n\n",
MAX_ATTEMPTS);

// Loop until the user runs out of attempts or guesses correctly
while (attempt <= MAX_ATTEMPTS && !guessed_correctly) {
    printf("Attempt %d of %d - Enter your guess: ", attempt, MAX_ATTEMPTS);

    // Read the integer input from the user
    if (scanf("%d", &guess) != 1) {
        printf("Invalid input. Please enter a valid integer.\n");
        break; // Exit if non-numeric input is provided
    }

    // Check the user's guess against the secret number
    if (guess == SECRET) {
        printf("Congratulations! You guessed the correct number!\n");
        guessed_correctly = 1; // Set flag to true to stop the loop
    } else if (guess < SECRET) {
        printf("Too low!\n\n");
    } else {
        printf("Too high!\n\n");
    }

    attempt++; // Increment the attempt counter
}

// Display final outcome message
if (guessed_correctly) {
    printf("\nYOU WIN! Great job.\n");
} else {
    printf("\nYOU LOSE! Out of attempts. The secret number was %d.\n", SECRET);
}

return 0;
}

```

Analysis:

This code is mostly correct and is easy to understand because it uses clear variable names, comments, and constants for the secret number and maximum attempts. However, there is a small error in the first line: `#include <stdio.t>` should be `#include <stdio.h>` for the program to compile. Its execution time is $O(1)$ because the loop can run at most three times, and its space

complexity is $O(1)$ because it only uses a few variables. The code is also maintainable because the constants make it easy to change the secret number or number of attempts. One other small issue is that if the user enters invalid input, the program immediately ends instead of allowing them to try again.